

```

import { useState, useEffect } from "react";
import { createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut, onA
import { collection, addDoc, doc, setDoc, getDoc, query, orderBy, where, onSnapsh
import { auth, db } from "../firebase";

const PRATICIENNE_EMAIL = "lamenaturo@gmail.com";

const TI = [
  { key: "complements", label: "Complements", question: "Tu as pris tes complemen
  { key: "sommeil", label: "Sommeil", question: "Comment tu as dormi ?" },
  { key: "cycle", label: "Cycle", question: "Ou en es-tu dans ton cycle ?" },
  { key: "digestion", label: "Digestion", question: "Comment etait ta digestion ?
  { key: "energie", label: "Energie", question: "Ton niveau d energie ?" },
  { key: "douleurs", label: "Douleurs", question: "Des douleurs cette semaine ?"
  { key: "humeur", label: "Humeur", question: "Tu etais comment emotionnellement
  { key: "alimentation", label: "Alimentation", question: "Tu as mange comment ce
];

const SC = [
  { v: 1, label: "Pas top", color: "#C4614A" },
  { v: 2, label: "Difficile", color: "#D4906A" },
  { v: 3, label: "Moyen", color: "#C8B86A" },
  { v: 4, label: "Bien", color: "#7BAF8C" },
  { v: 5, label: "Super bien", color: "#5BA08A" },
];

const bg = "#0c0f0e";
const sf = "rgba(255,255,255,0.04)";
const bd = "rgba(255,255,255,0.08)";
const ac = "#7EC8A0";
const ad = "rgba(126,200,160,0.15)";
const ab = "rgba(126,200,160,0.3)";
const wm = "#C8956C";
const wd = "rgba(200,149,108,0.12)";
const tx = "rgba(255,255,255,0.88)";
const tm = "rgba(255,255,255,0.45)";
const td = "rgba(255,255,255,0.22)";

const is = { width: "100%", background: sf, border: "1px solid " + bd, borderRadi

function btn(v) {
  const base = { padding: "11px 22px", borderRadius: 10, border: "none", cursor:
  if (v === "primary") return { ...base, background: ac, color: "#0c0f0e" };
  if (v === "warm") return { ...base, background: wm, color: "#0c0f0e" };

```

```

    return { ...base, background: sf, color: tm, border: "1px solid " + bd };
}

```

```

function SD({ value, size }) {
  size = size || 36;
  const s = SC.find(x => x.v === value);
  return (
    <div style={{ width: size, height: size, borderRadius: "50%", background: s ?
      {value || "-"}
    }>
    </div>
  );
}

```

```

function Auth({ onLogin }) {
  const [mode, setMode] = useState("login");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [prenom, setPrenom] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [resetSent, setResetSent] = useState(false);

```

```

  const login = async () => {
    setError(""); setLoading(true);
    try {
      const c = await signInWithEmailAndPassword(auth, email, password);
      const d = await getDoc(doc(db, "users", c.user.uid));
      onLogin({ uid: c.user.uid, email, prenom: d.data() ? d.data().prenom : "",
    } catch (e) {
      setError("Email ou mot de passe incorrect.");
    }
    setLoading(false);
  };

```

```

  const register = async () => {
    setError(""); setLoading(true);
    if (!email || !password || !prenom) { setError("Remplis tous les champs."); s
    try {
      const c = await createUserWithEmailAndPassword(auth, email, password);
      await setDoc(doc(db, "users", c.user.uid), { prenom, email, role: "cliente"
      onLogin({ uid: c.user.uid, email, prenom, role: "cliente" });
    } catch (e) {
      if (e.code === "auth/email-already-in-use") setError("Compte deja existant.
      else if (e.code === "auth/weak-password") setError("Mot de passe trop court
      else setError("Erreur creation compte.");
    }
    setLoading(false);
  };

```

```

};

const reset = async () => {
  if (!email) { setError("Entre ton email d abord."); return; }
  setError(""); setLoading(true);
  try { await sendPasswordResetEmail(auth, email); setResetSent(true); }
  catch (e) { setError("Email introuvable."); }
  setLoading(false);
};

return (
  <div style={{ minHeight: "100vh", display: "flex", alignItems: "center", justify
    <div style={{ width: "100%", maxWidth: 400 }}>
      <div style={{ textAlign: "center", marginBottom: 36 }}>
        <p style={{ color: tx, fontSize: 24, fontWeight: 700, fontFamily: "seri
        <p style={{ color: td, fontSize: 13 }}>Ton espace de suivi personnalis
      </div>
      <div style={{ background: sf, borderRadius: 16, border: "1px solid " + bd
        <div style={{ display: "flex", gap: 4, background: "rgba(255,255,255,0.
          <button onClick={() => { setMode("login"); setError(""); }} style={{
          <button onClick={() => { setMode("register"); setError(""); }} style=
        </div>
        <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
          {mode === "register" && (
            <div>
              <label style={{ color: td, fontSize: 11, textTransform: "uppercas
              <input value={prenom} onChange={e => setPrenom(e.target.value)} p
            </div>
          )}
        </div>
        <div>
          <label style={{ color: td, fontSize: 11, textTransform: "uppercase"
          <input value={email} onChange={e => setEmail(e.target.value)} place
        </div>
        <div>
          <label style={{ color: td, fontSize: 11, textTransform: "uppercase"
          <input value={password} onChange={e => setPassword(e.target.value)}
        </div>
        {error && <div style={{ color: "#D4826A", fontSize: 13, background: "
        {resetSent && <div style={{ color: ac, fontSize: 13, background: ad,
        <button onClick={mode === "login" ? login : register} disabled={loadi
        {mode === "login" && <button onClick={reset} style={{ background: "no
      </div>
    </div>
  </div>
</div>
);
}

```

```

function Cliente({ user, onLogout }) {
  const [entries, setEntries] = useState([]);
  const [messages, setMessages] = useState([]);
  const [view, setView] = useState("home");
  const [form, setForm] = useState({ scores: {}, humeur: "", confidences: "" });
  const [saved, setSaved] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const q = query(collection(db, "entries"), where("userId", "==", user.uid),
    const u = onSnapshot(q, s => { setEntries(s.docs.map(d => ({ id: d.id, ...d.d
    const q2 = query(collection(db, "messages"), where("toUId", "==", user.uid),
    const u2 = onSnapshot(q2, s => setMessages(s.docs.map(d => ({ id: d.id, ...d.
    return () => { u(); u2(); });
  }, [user.uid]));

  const wk = () => {
    const n = new Date();
    const s = new Date(n);
    s.setDate(n.getDate() - n.getDay() + 1);
    const mois = ["jan", "fev", "mar", "avr", "mai", "jun", "jul", "aou", "sep",
    return "Semaine du " + s.getDate() + " " + mois[s.getMonth()];
  };

  const submit = async () => {
    await addDoc(collection(db, "entries"), { userId: user.uid, userEmail: user.
    setForm({ scores: {}, humeur: "", confidences: "" });
    setSaved(true);
    setTimeout(() => { setSaved(false); setView("home"); }, 1800);
  };

  const lm = messages[messages.length - 1];
  if (loading) return <div style={{ color: td, padding: 40, textAlign: "center" }

  return (
    <div style={{ minHeight: "100vh", background: bg, padding: "28px 20px", fontF
      <div style={{ maxWidth: 600, margin: "0 auto" }}>
        <div style={{ display: "flex", justifyContent: "space-between", alignItem
          <div>
            <div style={{ color: ac, fontSize: 11, textTransform: "uppercase", le
              <h1 style={{ fontFamily: "serif", fontSize: 26, color: tx, fontWeight
            </div>
            <button onClick={onLogout} style={{ ...btn("ghost"), fontSize: 12, padd
          </div>
          {lm && (
            <div style={{ background: wd, border: "1px solid rgba(200,149,108,0.25)

```

```

        <div style={{ color: wm, fontSize: 11, textTransform: "uppercase", le
        <p style={{ color: tx, fontSize: 15, lineHeight: 1.6 }}>{lm.text}</p>
    </div>
  )}
  {view === "home" && (
    <div style={{ display: "flex", flexDirection: "column", gap: 12 }}>
      <button onClick={() => setView("new")} style={{ background: ad, borde
        <div>
          <div style={{ color: ac, fontWeight: 700, fontSize: 16, marginBot
            <div style={{ color: tm, fontSize: 13 }}>{wk()}</div>
          </div>
          <span style={{ fontSize: 22 }}>{">"</span>
        </button>
        {entries.length > 0 && (
          <button onClick={() => setView("history")} style={{ background: sf,
            <div>
              <div style={{ fontWeight: 600, fontSize: 15, marginBottom: 3 }}
                <div style={{ color: tm, fontSize: 13 }}>{entries.length} semai
              </div>
              <span style={{ fontSize: 20 }}>{">"</span>
            </button>
          )}
        </div>
      )}
    {view === "new" && (
      <div>
        <button onClick={() => setView("home")} style={{ ...btn("ghost"), fon
        <h2 style={{ fontFamily: "serif", fontSize: 20, color: tx, marginBott
        <p style={{ color: tm, fontSize: 13, marginBottom: 24 }}>{wk()}</p>
        {TI.map(item => (
          <div key={item.key} style={{ marginBottom: 20 }}>
            <div style={{ color: tx, fontWeight: 600, fontSize: 15, marginBot
              <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
                {SC.map(s => (
                  <button key={s.v} onClick={() => setForm(f => ({ ...f, scores
                    )}}
                </div>
              </div>
            </div>
          )}}
        <div style={{ marginBottom: 16 }}>
          <label style={{ color: td, fontSize: 11, textTransform: "uppercase"
            <textarea value={form.humeur} onChange={e => setForm(f => ({ ...f,
          </div>
          <div style={{ marginBottom: 24 }}>
            <label style={{ color: td, fontSize: 11, textTransform: "uppercase"
              <textarea value={form.confidences} onChange={e => setForm(f => ({ .
            </div>

```

```

        {saved ? <div style={{ background: ad, border: "1px solid " + ab, bor
        </div>
      }}
      {view === "history" && (
        <div>
          <button onClick={() => setView("home")} style={{ ...btn("ghost"), fon
          <h2 style={{ fontFamily: "serif", fontSize: 20, color: tx, marginBott
          {[...entries].reverse().map(e => {
            const vs = TI.map(i => e.scores && e.scores[i.key]).filter(Boolean)
            const avg = vs.length ? (vs.reduce((a, b) => a + b, 0) / vs.length)
            return (
              <div key={e.id} style={{ background: sf, borderRadius: 12, border
                <div style={{ display: "flex", justifyContent: "space-between",
                  <div>
                    <div style={{ color: tx, fontWeight: 600 }}>{e.weekLabel}</
                    <div style={{ color: td, fontSize: 12 }}>{new Date(e.date).
                  </div>
                    {avg && <SD value={Math.round(avg)} size={40} />}
                  </div>
                    {e.confidences && <div style={{ background: "rgba(255,255,255,0
                </div>
              )};
            )}}
          </div>
        )}
      </div>
    </div>
  );
}

```

```

function Praticienne({ user, onLogout }) {
  const [clients, setClients] = useState([]);
  const [selected, setSelected] = useState(null);
  const [entries, setEntries] = useState([]);
  const [messages, setMessages] = useState([]);
  const [newMsg, setNewMsg] = useState("");
  const [loading, setLoading] = useState(true);
  const [sending, setSending] = useState(false);

  useEffect(() => {
    const q = query(collection(db, "users"), where("role", "==", "cliente"));
    const u = onSnapshot(q, s => { setClients(s.docs.map(d => ({ uid: d.id, ...d.
    return u;
  }, []));

  const select = c => {
    setSelected(c); setNewMsg("");
  }
}

```

```

const q = query(collection(db, "entries"), where("userId", "==", c.uid), or
onSnapshot(q, s => setEntries(s.docs.map(d => ({ id: d.id, ...d.data() })))));
const q2 = query(collection(db, "messages"), where("toUid", "==", c.uid), or
onSnapshot(q2, s => setMessages(s.docs.map(d => ({ id: d.id, ...d.data() }))))
};

```

```

const send = async () => {
  if (!newMsg.trim()) return;
  setSending(true);
  await addDoc(collection(db, "messages"), { toUid: selected.uid, toEmail: sele
  setNewMsg(""); setSending(false);
};

```

```

if (loading) return <div style={{ color: td, padding: 40, textAlign: "center" }

```

```

return (
  <div style={{ minHeight: "100vh", background: bg, fontFamily: "sans-serif" }}
    <div style={{ maxWidth: 900, margin: "0 auto", padding: "28px 20px" }}>
      <div style={{ display: "flex", justifyContent: "space-between", alignItem
        <div>
          <div style={{ color: wm, fontSize: 11, textTransform: "uppercase", le
            <h1 style={{ fontFamily: "serif", fontSize: 26, color: tx, fontW
          </div>
          <button onClick={onLogout} style={{ ...btn("ghost"), fontSize: 12, padd
        </div>
        <div>
          <h2 style={{ color: tx, fontSize: 16, fontWeight: 600, marginBottom:
          <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>
            {clients.map(c => (
              <button key={c.uid} onClick={() => select(c)} style={{ background
                <div>
                  <div style={{ fontWeight: 600, fontSize: 15 }}>{c.prenom}</di
                  <div style={{ color: td, fontSize: 12 }}>{c.email}</div>
                </div>
                  <span style={{ color: td, fontSize: 20 }}>{">"</span>
                </button>
              )}}
            </div>
          </div>
        ) : (
          <div>
            <button onClick={() => setSelected(null)} style={{ ...btn("ghost"), f
            <h2 style={{ fontFamily: "serif", fontSize: 22, color: tx, fontW
            <div style={{ color: td, fontSize: 13, marginBottom: 24 }}>{selected.
            <h3 style={{ color: tm, fontSize: 12, textTransform: "uppercase", let
              {entries.length === 0

```

```

? <div style={{ color: td, background: sf, borderRadius: 12, paddin
: [...entries].reverse().map(e => {
  const vs = TI.map(i => e.scores && e.scores[i.key]).filter(Boolea
  const avg = vs.length ? (vs.reduce((a, b) => a + b, 0) / vs.lengt
  return (
    <div key={e.id} style={{ background: sf, borderRadius: 12, bord
      <div style={{ display: "flex", justifyContent: "space-between
        <div>
          <div style={{ color: tx, fontWeight: 600 }}>{e.weekLabel}
          <div style={{ color: td, fontSize: 12 }}>{new Date(e.date
        </div>
        {avg && <SD value={Math.round(avg)} size={40} />}
      </div>
      {e.humeur_libre && <div style={{ background: "rgba(126,200,16
      {e.confidences && <div style={{ background: "rgba(200,149,108
    </div>
  );
})
}
<h3 style={{ color: tm, fontSize: 12, textTransform: "uppercase", let
{messages.map(m => (
  <div key={m.id} style={{ background: wd, border: "1px solid rgba(20
    <p style={{ color: tx, fontSize: 14, lineHeight: 1.6 }}>{m.text}<
    <div style={{ color: td, fontSize: 11, marginTop: 6 }}>{new Date(
    </div>
  )))
  <textarea value={newMsg} onChange={e => setNewMsg(e.target.value)} pl
  <button onClick={send} disabled={sending || !newMsg.trim()} style={{
</div>
  ))
</div>
</div>
);
}

export default function App() {
  const [user, setUser] = useState(null);
  const [checking, setChecking] = useState(true);

  useEffect(() => {
    const u = onAuthStateChanged(auth, async fw => {
      if (fw) {
        const d = await getDoc(doc(db, "users", fw.uid));
        setUser({ uid: fw.uid, email: fw.email, prenom: d.data() ? d.data().prenom
      } else {
        setUser(null);
      }
    }
  )
}

```

```

        setChecking(false);
    });
    return u;
}, []);

const logout = async () => { await signOut(auth); setUser(null); };

if (checking) return <div style={{ minHeight: "100vh", background: "#0c0f0e", d

return (
    <>
        <style>{"* { box-sizing: border-box; margin: 0; padding: 0; } body { backgr
        {!user ? <Auth onLogin={setUser} /> : user.role === "praticienne" ? <Pratic
    </>
);
}

```